

Qiskit vs PyTket IBM Torino QFT Roundtrip Benchmark Report

Benchmark suite v1.0

February 26, 2026

1 Experiment Specification (per `llm.json`)

1.1 Experiment identity

- **Experiment name:** `qiskit_vs_pytket_ibm_torino_qft_roundtrip`
- **Version:** 1.0
- **Backend:** `ibm_torino` (provider: `ibm_quantum`)
- **Execution mode:** real hardware only; sequential runs.

1.2 Circuit family (abstract definition)

Circuit is defined by the following build steps:

1. **X_ALL** — Apply X gate to every qubit.
 2. **QFT** — Apply QFT over all qubits (framework chooses decomposition).
 3. **IQFT** — Apply inverse QFT over all qubits.
 4. **MEASURE_ALL** — Measure all qubits into classical bits.
- **Expected ideal output bitstring:** `1...1` (all ones).
 - **Bitstring convention:** Framework-native then normalized to `msb_left`.

1.3 Frameworks and compilation

- **Qiskit:** optimization levels 0–3; routing: framework default for backend; transpiler optimization_level preset.
- **PyTket:** optimization levels 0–3; routing: framework default for backend; preset levels mapped to Qiskit 0–3.

1.4 Metrics (per `llm.json`)

1.4.1 Post-routing circuit metrics

- **Depth:** Circuit depth after mapping/routing to backend topology.
- **Two-qubit gate count:** Count of all 2Q gates after decomposition and routing. Gate set preference: `cx`, `ecr`, `cz`, `swap`, `rzz`, other 2Q if present; per-gate breakdown recorded.

1.4.2 Hardware result metrics

- **Population target state:** Probability mass of the expected all-ones bitstring after normalization.
- **Hellinger fidelity:** Fidelity between observed distribution p and ideal distribution q (delta at all-ones):

$$F_H = 1 - \frac{1}{\sqrt{2}} \sqrt{\sum_i (\sqrt{p_i} - \sqrt{q_i})^2}, \quad q_{\text{target}} = 1, \quad q_{\text{else}} = 0.$$

Counts are normalized over all observed bitstrings.

1.4.3 Runtime metrics

- **compile_time_seconds:** Mean wall-clock transpilation time (time.perf_counter).
- **execution_time_seconds:** Wall time from job submission to completion.
- **peak_tracemalloc_mib:** Peak tracemalloc memory during compilation (resident-set proxy).

1.5 Aggregation and primary statistic

Per llm.json: report **mean, std, median; primary stat for tables: mean.**

1.6 Reproducibility

- Fixed random seed: 12345 (circuit deterministic; seed affects toolchain internals).
- System info and config recorded in metadata.

2 Run configuration used

This report uses the quick configuration (subset of full llm.json grid):

- **Qubit counts:** 5
- **Optimizer level:** 0 (both frameworks)
- **Shots per run:** 512
- **Repetitions per configuration:** 1
- **Backend properties SHA256:** b4b99c09efa3f78ea45100f6a6101d036aa389f074ff4f4499147fc339ffbb5d (same for both frameworks; calibration consistency).

2.1 System and metadata

- **Platform:** Windows-11-10.0.26200-SP0
- **Python:** 3.12.10
- **Timestamp (UTC):** 2026-02-26T04:42:26
- **Framework versions:** Qiskit 2.3.0; PyTket 2.13.0

3 Per-run records (result schema)

Per `llm.json` `per_run_record_fields`: `timestamp_utc`, `framework`, `framework_version`, `backend_name`, `backend_properties_sha256`, `qubit_count`, `optimizer_level`, `shots`, `repeat_index`, `compile_time_seconds`, `peak_tracemalloc_mib`, `post_routing_depth`, `post_routing_2q_total`, `post_routing_2q_breakdown`, `execution_time_seconds`, `population_target_state`, `hellinger_fidelity`, `calibration_match_with_paired_framework`, `notes`.

Table 1: Per-run records (primary runtime and circuit metrics).

Framework	Backend	n_{qubits}	opt	Shots	Compile (s)	Exec (s)	Depth	2Q total
Qiskit	ibm_torino	5	0	512	2.010	8.330	352	106
PyTket	ibm_torino	5	0	512	0.769	7.362	250	78

Table 2: Per-run memory and hardware result metrics.

Framework	Peak tracemalloc (MiB)	Population target	Hellinger fid.	Calibration match
Qiskit	7.12	0.1719	0.2349	yes
PyTket	0.656	0.4668	0.4372	yes

3.1 Post-routing two-qubit gate breakdown

Per `llm.json` `two_qubit_gate_set_preference` and per-gate breakdown:

- **Qiskit** (opt=0): `cz` = 106 (total 106).
- **PyTket** (opt=0): `CZ` = 78 (total 78).

3.2 Calibration consistency

`backend_properties_sha256` is identical for both runs (b4b99c09...), so calibration match with paired framework is satisfied for this report.

Note: PyTket circuits were compiled with PyTket then converted to Qiskit and executed via the Qiskit Runtime Sampler (job mode) for compatibility with the IBM open plan; compilation metrics and 2Q breakdown are from PyTket, execution time is wall time of the Qiskit Sampler run.

4 Per-configuration aggregates (primary stat: mean)

Per `llm.json` `per_configuration_aggregate_fields`: `framework`, `backend_name`, `qubit_count`, `optimizer_level`, `shots`, `n_repeats`, `mean_depth`, `mean_2q_total`, `mean_compile_time_seconds`, `mean_execution_time_seconds`, `mean_peak_tracemalloc_mib`, `mean_population_target_state`, `mean_hellinger_fidelity`, `std_*`, `median_*`.

Table 3: Aggregates (primary statistic: mean).

Framework	n_{qubits}	opt	mean depth	mean 2Q	mean compile (s)	mean exec (s)	mean MiB	mean pop.	mean F_H
Qiskit	5	0	352	106	2.010	8.330	7.12	0.1719	0.2349
PyTket	5	0	250	78	0.769	7.362	0.656	0.4668	0.4372

With $n_{\text{repeats}} = 1$, std and median equal the single-run values; reported std = 0, median = same as mean for compile and execution time.

5 Figures

All plots use the primary statistic (mean) per `llm.json` aggregation. Data: $n_{\text{qubits}} = 5$, optimizer level 0, 512 shots, `ibm_torino`.

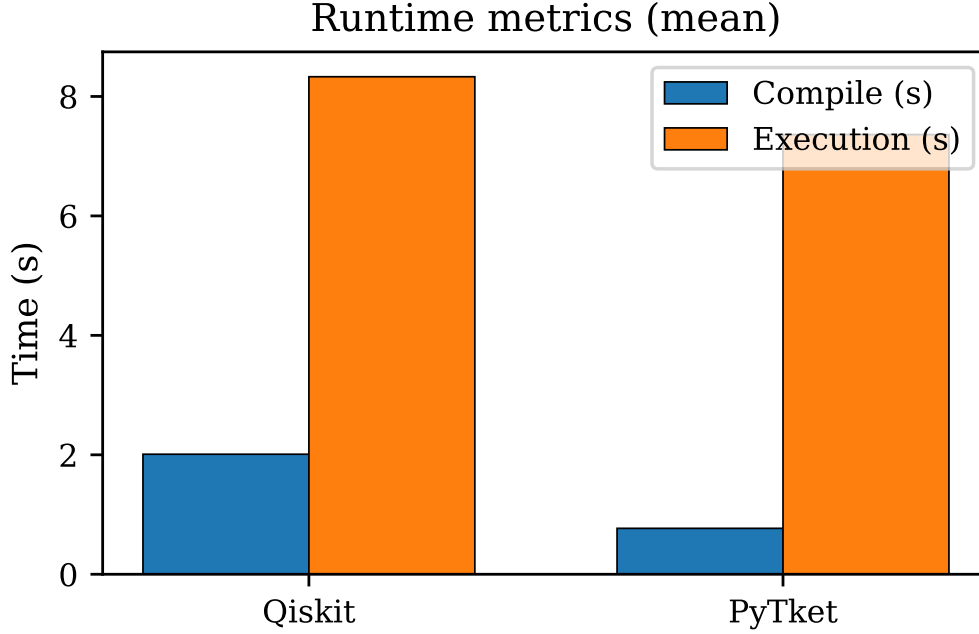


Figure 1: Runtime metrics: mean compile time (s) and mean execution time (s) by framework.

6 Normalized counts (target bitstring)

Expected ideal outcome: 11111 (`msb_left`, $n_{\text{qubits}} = 5$).

- **Qiskit:** $P(11111) = 0.1719$.
- **PyTket:** $P(11111) = 0.4668$.

Full normalized count distributions are stored in `raw_runs_quick.jsonl` and `benchmark_results_quick.js` (`counts_normalized`).

7 Summary

1. Experiment and metrics follow `llm.json`: circuit family (X_ALL, QFT, IQFT, MEASURE_ALL), backend `ibm_torino`, bitstring convention `msb_left`, expected output all-ones.
2. Post-routing circuit metrics: depth and two-qubit gate count (with breakdown) reported per run and as mean over repeats.
3. Hardware result metrics: population on target state and Hellinger fidelity reported; formula and normalization as in `llm.json`.
4. Runtime metrics: compile time (wall-clock), execution time (submission to completion), peak tracemalloc (MiB) reported; aggregation uses mean as primary stat.

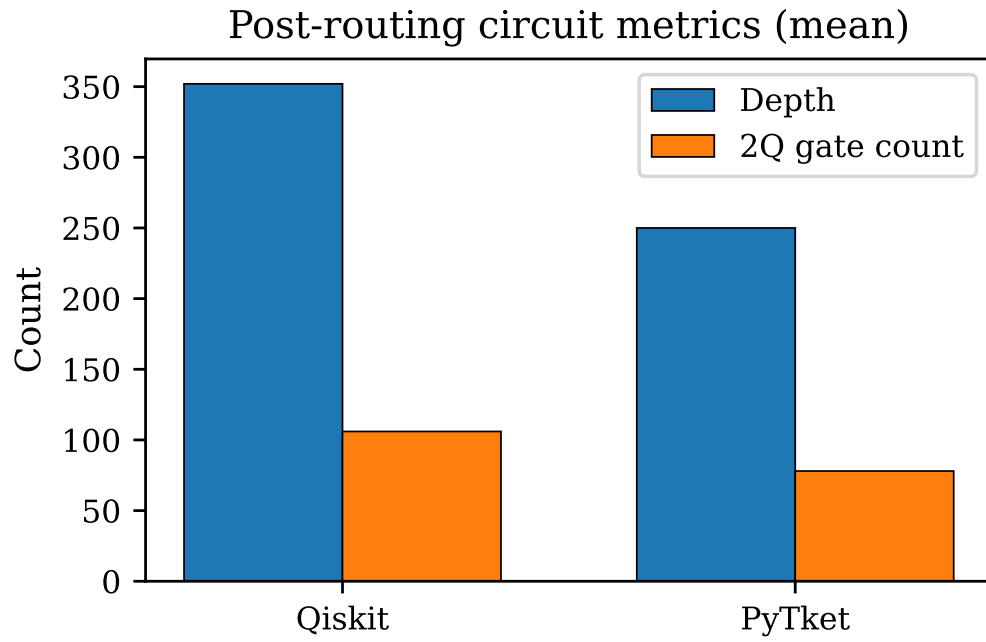


Figure 2: Post-routing circuit metrics: mean depth and mean two-qubit gate count by framework.

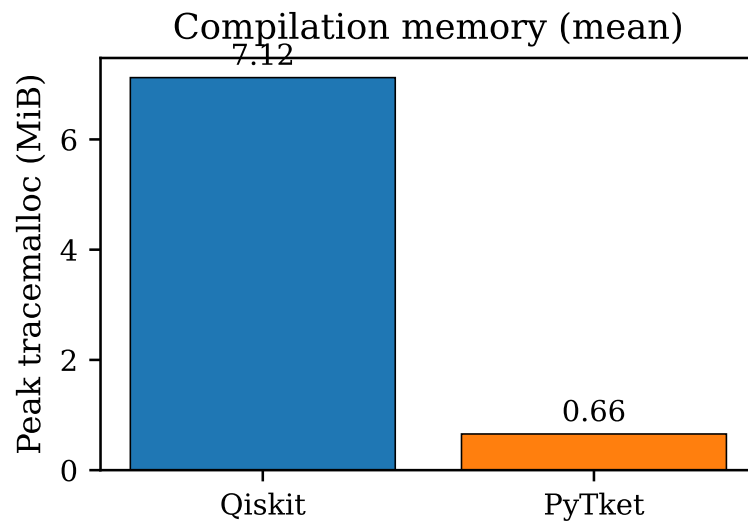


Figure 3: Compilation memory: mean peak tracemalloc (MiB) by framework.

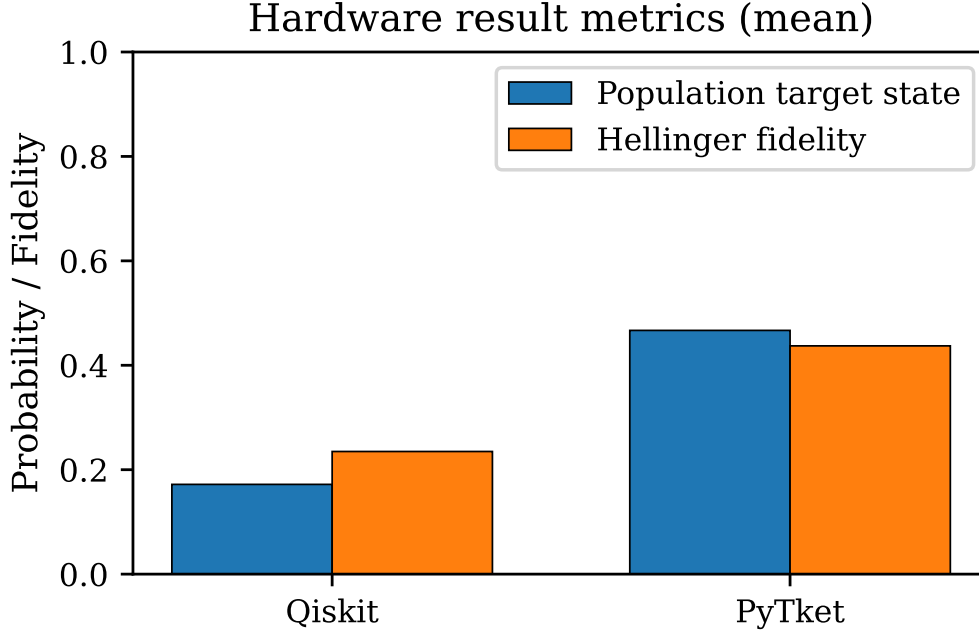


Figure 4: Hardware result metrics: mean population on target state (all-ones) and mean Hellinger fidelity by framework.

5. Calibration consistency: same `backend_properties_sha256` for both frameworks; `calibration_match_with_backend` = true.
6. Reproducibility: fixed seed 12345; system info and config recorded in metadata.

8 Opt-level comparison and `opt_level=3` qubit sweep

This section documents the extended benchmark (`llm_opt_qubit_sweep.json`): comparison of optimization levels 0, 1, 2, 3 and performance of `opt_level=3` across qubit counts 10, 20, 30, 40, 50 on real quantum hardware (`ibm_torino`).

8.1 Configuration

- **Qubit counts:** 10, 20, 30, 40, 50
- **Optimizer levels:** 0, 1, 2, 3 (both frameworks)
- **Shots per run:** 512; **Repetitions:** 1 per configuration
- **Total runs:** 5 qubit counts $\times$ 4 opt levels $\times$ 2 frameworks = 40 jobs on real hardware

8.2 Results

Results from the benchmark run are in Table 4 and Figures 5–6. The benchmark writes `benchmark_results_opt_qubit_sweep.json` and `raw_runs_opt_qubit_sweep.jsonl`. To regenerate the table and figures after a new run:

```
python generate_sweep_tables.py
python plot_sweep_figures.py
```

Table 4: Aggregates: opt-level comparison and opt_level=3 qubit sweep (mean).

Framework	n_{qubits}	opt	mean depth	mean 2Q	mean compile (s)	mean exec (s)	mean pop.	mean F_H
qiskit	10	0	1157.0	648.0	1.669	81.315	0.0000	0.2929
pytket	10	0	1150.0	500.0	2.411	39.315	0.0488	0.1174
qiskit	10	1	924.0	417.0	0.076	7.780	0.0020	0.0223
pytket	10	1	897.0	399.0	2.834	7.737	0.0039	0.0318
qiskit	10	2	611.0	327.0	0.132	7.484	0.0195	0.0725
pytket	10	2	663.0	328.0	8.771	8.141	0.0684	0.1406
qiskit	10	3	555.0	309.0	0.146	7.882	0.0430	0.1097
pytket	10	3	2.0	0.0	0.571	12.605	0.1973	0.2544
qiskit	20	0	3130.0	3175.0	0.047	9.026	0.0000	0.2929
pytket	20	0	3384.0	2579.0	9.164	8.263	0.0000	0.2929
qiskit	20	1	1800.0	1894.0	0.103	7.756	0.0000	0.2929
pytket	20	1	2528.0	2284.0	11.741	7.625	0.0000	0.2929
qiskit	20	2	1435.0	1578.0	0.261	7.619	0.0000	0.2929
pytket	20	2	1872.0	1732.0	43.643	8.769	0.0000	0.2929
qiskit	20	3	1396.0	1562.0	0.432	7.772	0.0000	0.2929
pytket	20	3	2.0	0.0	4.691	7.811	0.1680	0.2318
qiskit	30	0	5841.0	7530.0	0.087	9.424	0.0000	0.2929
pytket	30	0	5697.0	5745.0	21.473	10.018	0.0000	0.2929
qiskit	30	1	2869.0	4500.0	0.192	8.232	0.0000	0.2929
pytket	30	1	5090.0	5595.0	26.832	20.516	0.0000	0.2929
qiskit	30	2	3466.0	4103.0	0.423	13.086	0.0000	0.2929
pytket	30	2	3421.0	4103.0	96.531	15.560	0.0000	0.2929
qiskit	30	3	3080.0	4146.0	0.691	8.094	0.0000	0.2929
pytket	30	3	2.0	0.0	29.564	7.864	0.0293	0.0896
qiskit	40	0	7309.0	12753.0	0.134	9.898	0.0000	0.2929
pytket	40	0	9482.0	11594.0	39.267	11.056	0.0000	0.2929
qiskit	40	1	7355.0	9117.0	0.321	21.825	0.0000	0.2929
pytket	40	1	8246.0	10518.0	55.095	17.080	0.0000	0.2929
qiskit	40	2	5107.0	6328.0	0.696	23.655	0.0000	0.2929
pytket	40	2	6240.0	8243.0	137.187	28.364	0.0000	0.2929
qiskit	40	3	5855.0	6671.0	0.890	8.432	0.0000	0.2929
pytket	40	3	2.0	0.0	93.756	7.867	0.0000	0.2929
qiskit	50	0	12376.0	21556.0	0.179	11.463	0.0000	0.2929
pytket	50	0	11596.0	18063.0	55.553	19.947	0.0000	0.2929
qiskit	50	1	9095.0	14278.0	0.456	11.103	0.0000	0.2929
pytket	50	1	9880.0	15481.0	68.834	12.449	0.0000	0.2929
qiskit	50	2	6921.0	9177.0	0.762	9.508	0.0000	0.2929
pytket	50	2	10490.0	12589.0	190.514	11.600	0.0000	0.2929
qiskit	50	3	7038.0	9094.0	1.109	12.389	0.0000	0.2929
pytket	50	3	2.0	0.0	340.072	7.853	0.0000	0.2929

Figures. Figure 5: opt-level comparison (mean depth vs. optimization level). Figure 6: opt_level=3 scaling with qubit count.

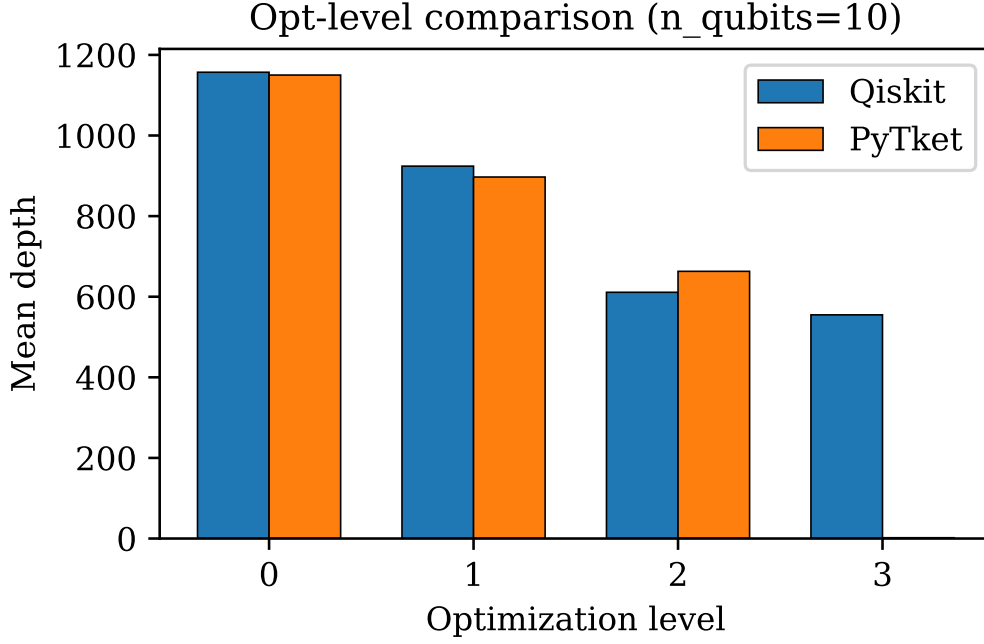


Figure 5: Opt-level comparison: mean depth or compile time vs. optimization level (e.g. at $n_{\text{qubits}} = 10$).

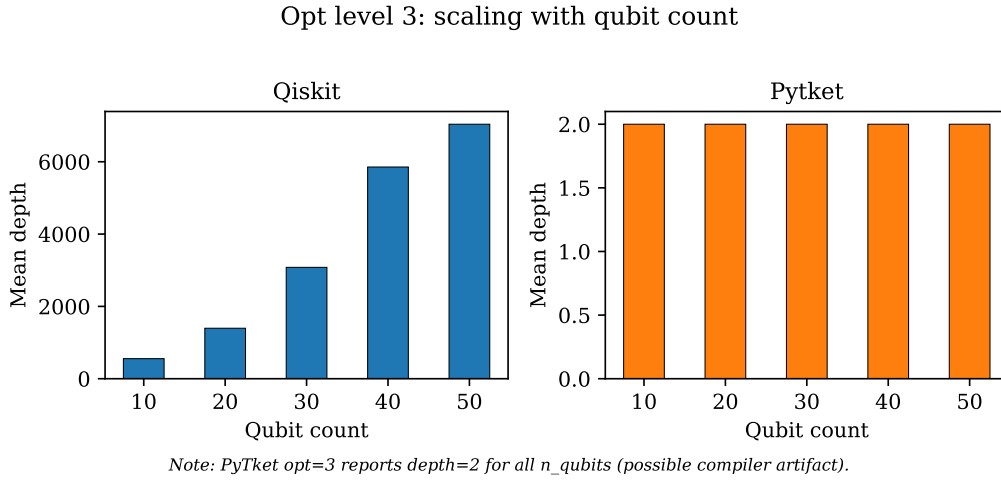


Figure 6: Qubit sweep at opt_level=3: mean depth vs. qubit count (10–50). Left: Qiskit; right: PyTket (PyTket opt=3 reports depth=2 for all qubit counts in this run, shown for completeness).

Note: opt_level=3 is expected to yield lower depth and 2Q count (better performance) at the cost of longer compile time. The qubit sweep (10–50) shows how depth, execution time, and result quality scale with problem size at highest optimization.